Лекция 6
Авторизация. Сессии

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Техническое задание

- Состав **ТЗ** на итоговую систему (использовать сплошную нумерацию внутри пунктов):

1. **цель**
2. **назначение** - краткое описание для чего, кто работает в системе
3. **задачи** - 12 задач (8 лабораторных-задач по бекенду/фронтенду, нативное приложение, демо в Pages, документация, репозиторий Git) с указанием ваших технологий по каждой задаче
4. **Функциональные требования** - список HTTP методов, далее список окон и какие действия для каких групп пользователей доступны. Указать, какие методы бэкенда при этом вызываются. Всего 14 подпунктов - HTTP методы, меню и 8-12 страниц приложения:

- гость: регистрация, аутентификация, главная, список услуг, одна услуга, 403 и 404
- создатель заявки: конструктор заявки, список заявок, личный кабинет
- <u>модератор: список услуг таблицей, редактирование/создание услуги</u>

5. **требования к аппаратному** обеспечению для сервера и клиента
6. **требования к программному** обеспечению с версиями для серверных компонентов и для клиента

1. Цель работы

Реализовать систему привязки участников ВОВ к архивным документам, включающую в себя веб-сервис, веб-приложение, десктопное приложение и выделенный сервис проверки подлинности документов.

2. Назначение

Система разработана для участников и архивистов архивной платформы Великой Отечественной войны. В данной платформе предусмотрен ограниченный доступ к архивным документам участников событий. Для получения разрешения на доступ к определенному документу участник обязан подать заявку через автоматизированную систему, указав выбранный документ. Система осуществляет создание, учет и отслеживание статуса заявок. Участники могут добавлять новые документы в архив, которые позже будут подтверждаться выделенным сервисом проверки подлинности. Архивисты имеют возможность принимать или отклонять заявки, а также вносить изменения в уже существующие документы и добавлять новые в архив.

3. Задачи:

3.1. → Разработать дизайн приложения.

3.2. → Создать базу данных в PostgreSQL.

3.3. → Создать веб-сервис на технологии Golang 1.20.

3.4. → Реализовать интерфейс гостя на технологии React.

3.5. → Развернуть веб-приложение React на Github Pages.

3.6. → Добавить авторизацию и аутентификацию в веб-сервис.

Оформление и ГОСТ

- Документацию оформляем по ГОСТ (1.5 интервал, 1.25 см отступ, 14 пт и тд)
- Требования по оформлению, ГОСТ, **TestVKR** необходимо искать на сайте с требованиями по ВКРБ (ссылка дана в репозитории)
- Отчет по ДЗ обязательно проверить через **TestVKR**

РПЗ ВКР должна быть **грамотно написана** и **правильно оформлена**. Она должна быть распечатана на одной стороне ***белого*** листа бумаги формата А4 (210х297 мм) шрифтом ***черного цвета Times New Roman размером 14 пунктов***, кроме фрагментов кода программ, для которых необходимо использовать шрифт ***Courier New***.

При выполнении РПЗ необходимо соблюдать ***равномерную плотность, контрастность и четкость изображения по всему документу***. В нем должны быть четкие, не расплывшиеся линии, буквы, цифры и знаки.

Разрешается использовать компьютерные возможности акцентирования внимания на определенных терминах, формулах, теоремах, применяя разное начертание шрифта.

Для переноса слов в тексте необходимо использовать автоматическую расстановку переносов.

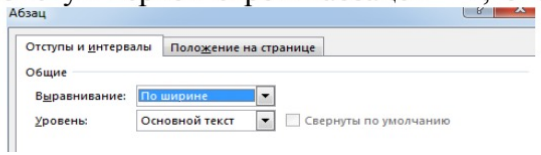
Ниже показаны примеры оформления текста РПЗ и различных структурных элементов.

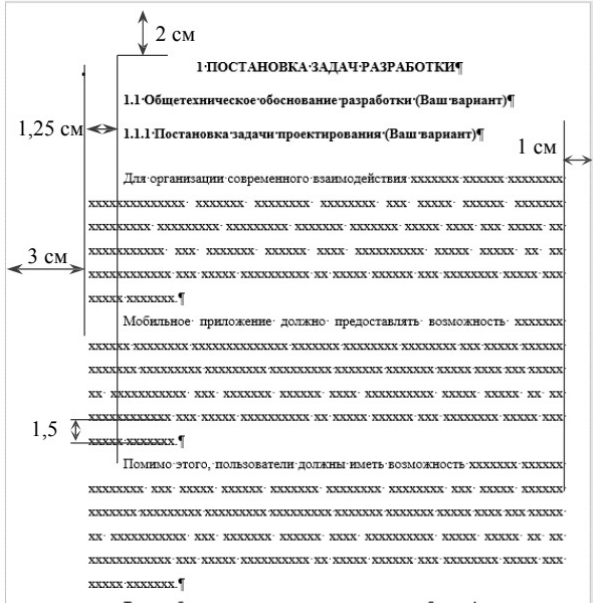
В РПЗ необходимо соблюдать следующие размеры полей страницы: левое – 3 см, правое – 1 см, нижнее – 2 см, верхнее – 2 см.

Колонтитулы (верхние и нижние) во всем документе устанавливаются - 1,25.

Верхний колонтитул должен быть пустой, в нижнем должна быть только одна строка, в центре которой, симметрично тексту, должен стоять номер страницы.

Выравнивание **ВСЕГО** текста (кроме заголовков и текста в таблицах) – по ширине, без отступов и интервалов по вертикали. Междустрочный интервал – 1,5. Отступ первой строки абзацев – 1,25 см.





https://drive.google.com/file/d/1bmQMZeqCY9WmmqQviDvA0rju7a1RgMh_/view

Расчет аппаратных требований *

- В системе работают 250 исследователей и 20 профессоров. Пусть исследователь формирует 70 заявок на расчёт

$$N_{requests/day} = 250 \times 70 = 17500 \text{ заявок/сутки}$$

- Вычислим RPS с учётом того, что каждый профессор даёт ещё по одному RPS из-за polling Вычислим RPS с учётом того, что каждый профессор даёт ещё по одному RPS из-за polling

$$N_{cores} = \frac{RPS}{RPS_{core}} \times k_{reserve} = \frac{28{,}51}{500} \times 1{,}5 \approx 1 \text{ ядро}$$

$$RAM = 4 \times N_{cores} = 4 \times 1 = 4 \text{ ГБ}$$

$$RPS = \frac{N_{requests/day} \times 14}{28800} + 20 \times 1 = \frac{17500 \times 14}{28800} + 20 \approx 28{,}51 \text{ запросов/с}$$

- Рассчитаем размеры записей в таблицы request_de_broglie_calculations (ид заявки и 2 пользователей – по 8 байт …

$$S_{rdbc} = 8 \times 3 + 8 \times 3 + 100 + 20 = 168 \text{ байт}$$

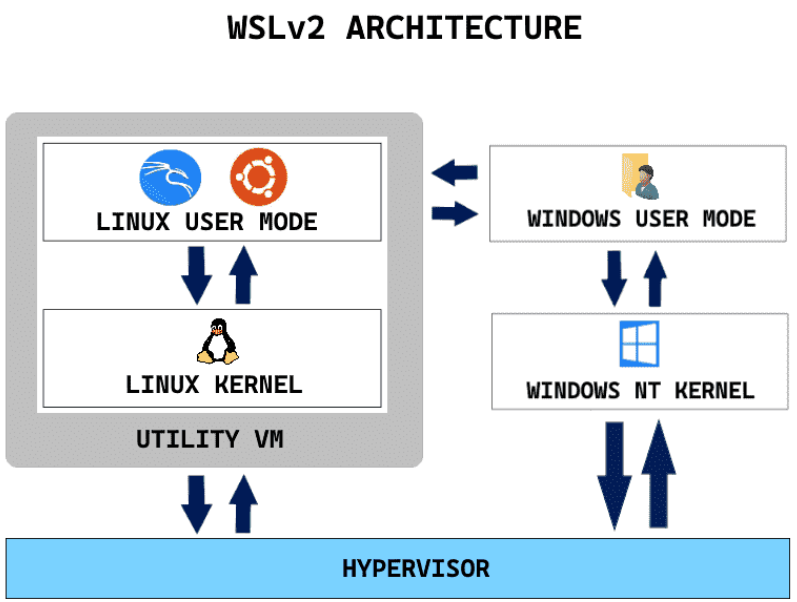
$$DB_{growth/year} = \frac{N_{year} \times (S_{rdbc} + 3 \times S_{dbc})}{1024 \times 1024 \times 1024} = \frac{6387500 \times (168 + 3 \times 56)}{1024 \times 1024 \times 1024} \approx 2 \text{ ГБ/год}$$

WSL - для Redis под Windows

- **Windows Subsystem for Linux** (WSL) — слой совместимости для запуска Linux-приложений (двоичных исполняемых файлов в формате ELF) в ОС Windows

Он понадобится чтобы запустить на Windows:

- Docker
- Kafka
- Redis



Redis *

REmote **DI**ctionary **S**erver, «удалённый серверный словарь»:

- Резидентная система управления базами данных
- Данные размещаются в оперативной памяти
- Механизмы снимков на дисках для постоянного хранения
- Структура данных ключ-значение, словаря
- Максимальная производительность на атомарных операциях
- Механизм подписок не гарантирует, что сообщение будет доставлено

Redis. Отличия от реляционных *

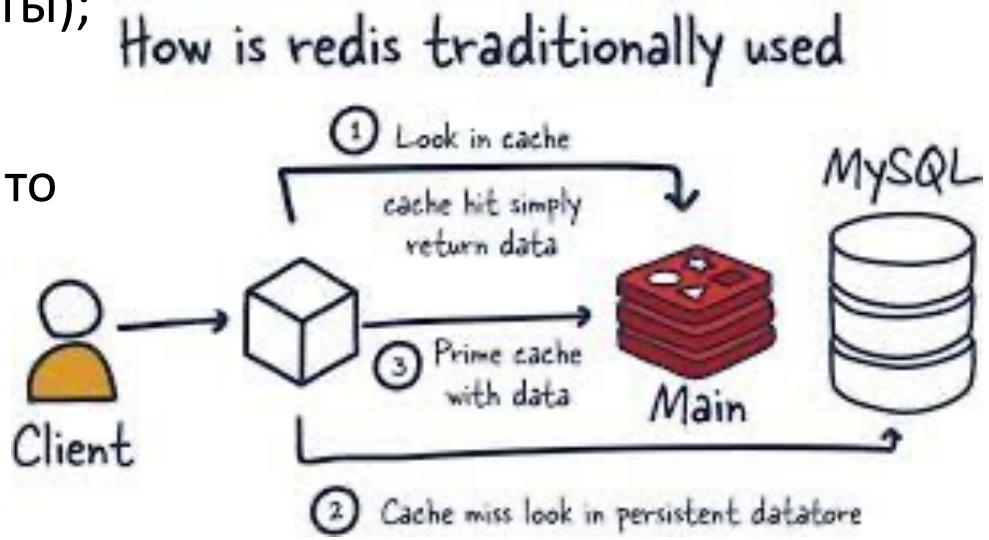
От реляционных баз Redis отличается:

- **более высокой производительностью** (благодаря хранению данных в оперативной памяти сервера, значительно увеличивается число выполняемых операций);
- **отсутствием языка SQL** (Lua-скрипты как альтернатива);
- **гибкостью** (данные находятся не в жёстких структурах (таблицах), а в более удобных (строки, списки, хеши, множества, сортированные множества), что облегчает работу программисту;
- **лучшей масштабируемостью**.

Однако Redis редко используется как основное хранилище в крупных системах, так как не удовлетворяет требованиям ACID, то есть не обеспечивает 100%-ной целостности данных.

Redis. Применение *

- для хранения пользовательских сессий (HTML-фрагменты веб-страниц или товары корзины интернет-магазина);
- для хранения промежуточных данных (поток сообщений на стене, голосовалки, таблицы результатов);
- как брокер сообщений (стратегия «издатель-подписчик» позволяет создавать новостные ленты, групповые чаты);
- как СУБД для небольших приложений, блогов;
- для кэширования данных из основного хранилища, что значительно снижает нагрузку на реляционную базу данных;
- для хранения «быстрых» данных — когда важны скорость и критичны задержки передачи (аналитика и анализ данных, финансовые и торговые сервисы).



Redis. Пример

- HSET — сохраняет значение по ключу
- создали объект person1 с двумя полями (name и age) и соответствующими значениями.

```
127.0.0.1:6379> HSET person1 name "Aleksey"
(integer) 1
127.0.0.1:6379> HSET person1 age 25
(integer) 1
```

- HGET — получение значения по ключу (для определённого поля)
- Получили значение поля name у ключа person1

```
127.0.0.1:6379> HGET person1 name
"Aleksey"
```

Redis

- Установить

```
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/redis-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://packages.redis.io/deb $(lsb_release

sudo apt-get update
sudo apt-get install redis
```

- Запустить

```
sudo service redis-server start
```

- Использовать

```
redis-cli
127.0.0.1:6379> ping
PONG
```

- Можем получить все ключи из Redis

```
keys *
```

- Или написать целый скрипт на языке Lua для получения активных сессий пользователей

```
EVAL "local key_name = 'example_key'; for iterated_value=0,4 do redis.call('hmset', KEYS[1],
```

Redis с NestJS

- Пропишем сокет запущенного Redis:

```
constructor() {
  this.redis = new Redis({
    host: 'localhost',
    port: 6379,
  });
```

- Далее создадим инстанс нашего хранилища сессий:

```
async create(sessionId: string, userId: number, ttlSec: number = 3600): Promise<void> {
  await this.redis.setex(`session:${sessionId}`, ttlSec, String(userId));
}
```

Аутентификация, Авторизация *

Аутентифика́ция (*authentication*) — процедура проверки подлинности, например:

- проверка подлинности пользователя путём сравнения введённого им пароля (для указанного логина) с паролем, сохранённым в базе данных пользовательских логинов;
- подтверждение подлинности электронного письма путём проверки цифровой подписи письма по открытому ключу отправителя;

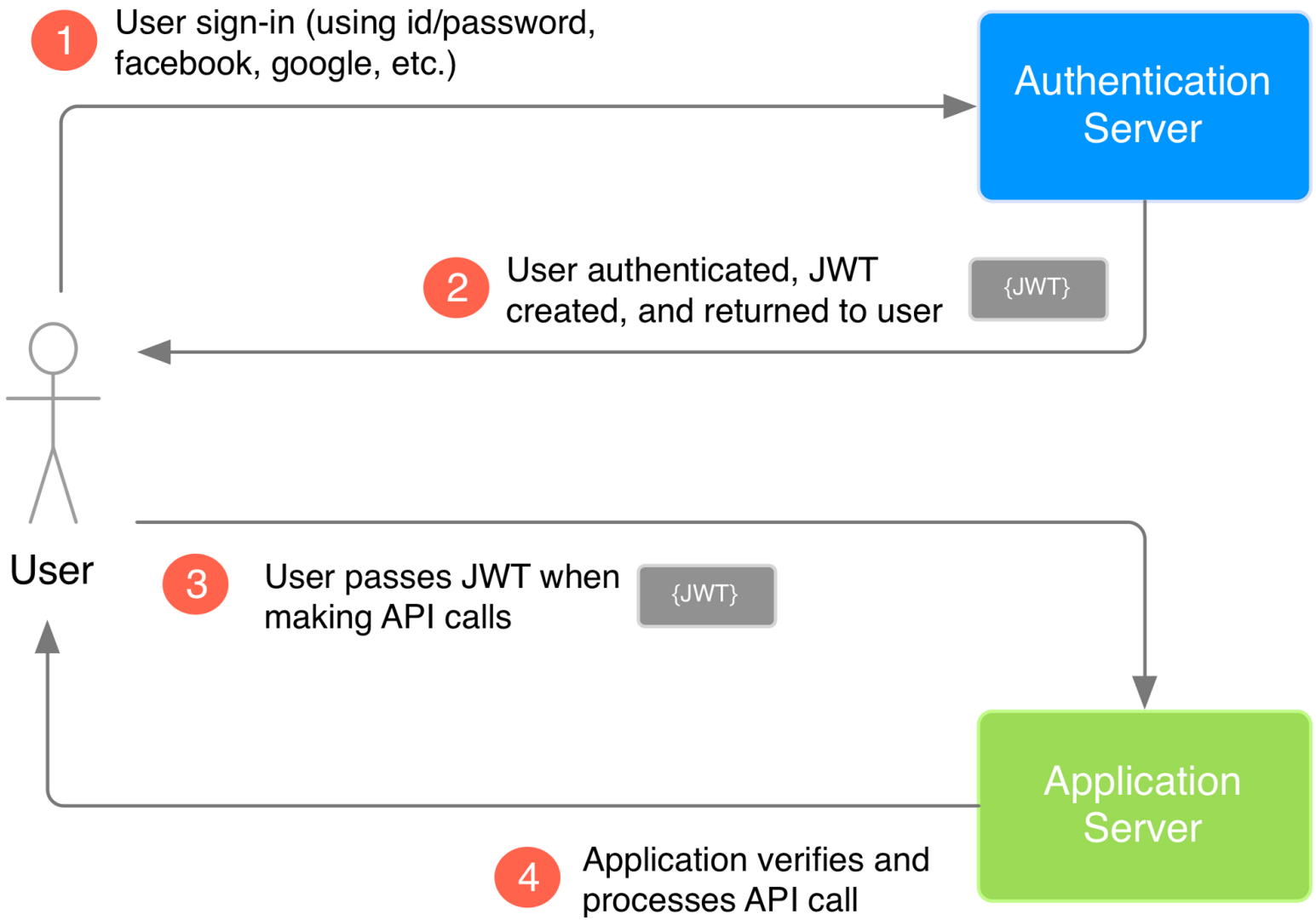
Идентификация — процедура, в результате выполнения которой для субъекта идентификации выявляется его идентификатор, однозначно определяющий этого субъекта в информационной системе (**Определяем** пользователя для заявки).

Авториза́ция (*authorization* «разрешение; уполномочивание») — предоставление определённому лицу или группе лиц прав на выполнение определённых действий; а также процесс проверки (подтверждения) данных прав при попытке выполнения этих действий (**гость** смотрит товар, **покупатель** добавляет в заказ, **продавец** - редактирует).

- Авторизация производит контроль доступа к различным ресурсам системы в процессе работы легальных пользователей после успешного прохождения ими аутентификации.

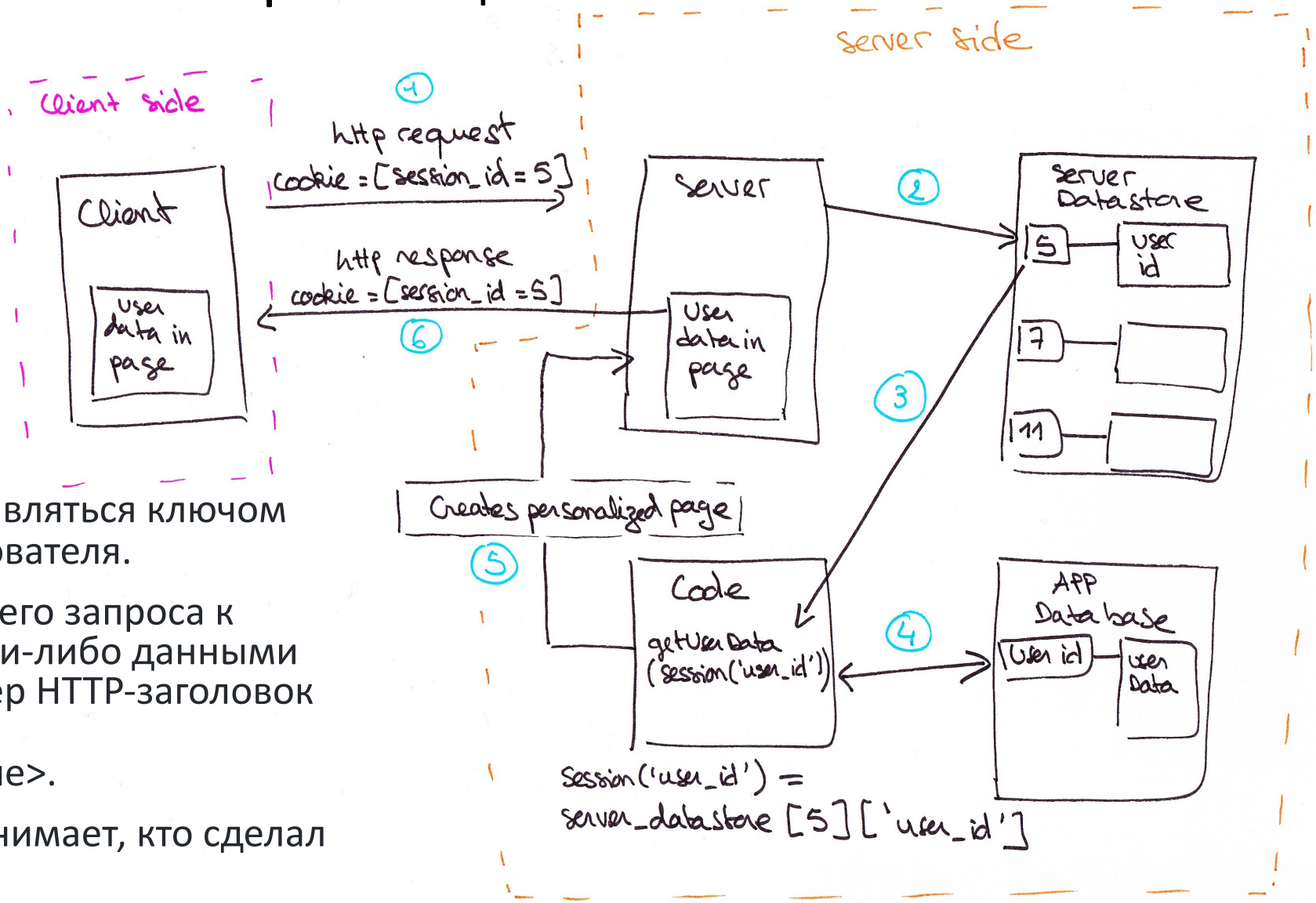
Схема аутентификации и авторизации *

- Схема приведена для JWT, но мы заменив Authentication Server на метод аутентификации получим общую схему
- Метод аутентификации получает логин и пароль и если они правильные (таблица пользователь), возвращает обратно id сессии или JWT
- Во время авторизации каждый наш метод из 3ей лабораторной (Application Server) проверяет правильность сессии/JWT для выполнения действий



Сессии – схема авторизации *

- При авторизации на сайте сервер отсылает в ответ HTTP-заголовок Set-Cookie (5), чтобы сохранить куки в браузере с уникальным идентификатором сессии («session identifier»).
- Это идентификатор будет являться ключом уникальным сессии пользователя.
- Во время любого следующего запроса к этому же серверу за какими-либо данными браузер посылает на сервер HTTP-заголовок Cookie (1), в которым в формате <ключ>=<значение>.
- Таким образом, сервер понимает, кто сделал запрос.

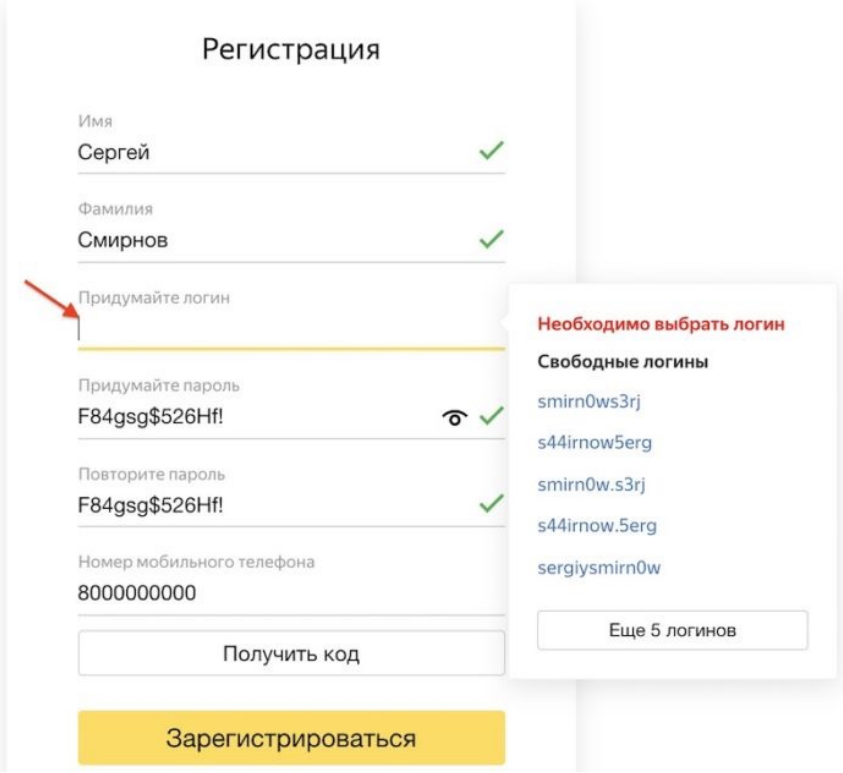


Куки *

- **Ку́ки** (*cookie*, букв. — «печенье») — небольшой фрагмент данных, отправленный веб-сервером и хранимый на компьютере пользователя
- Куки устанавливает сервер в **HTTP-ответе** в заголовке **Set-Cookie**
- Веб-клиент (обычно веб-браузер) всякий раз при попытке открыть страницу соответствующего сайта пересылает этот фрагмент данных веб-серверу в составе **HTTP-запроса** в заголовке **Cookie**.
- Применяется для сохранения данных на стороне пользователя

Пользователи

- Концептуально сущность пользователя должна содержать его личные данные, такие данные:
- номер телефона
- почта
- имя
- никнейм
- пароль
- и тд...



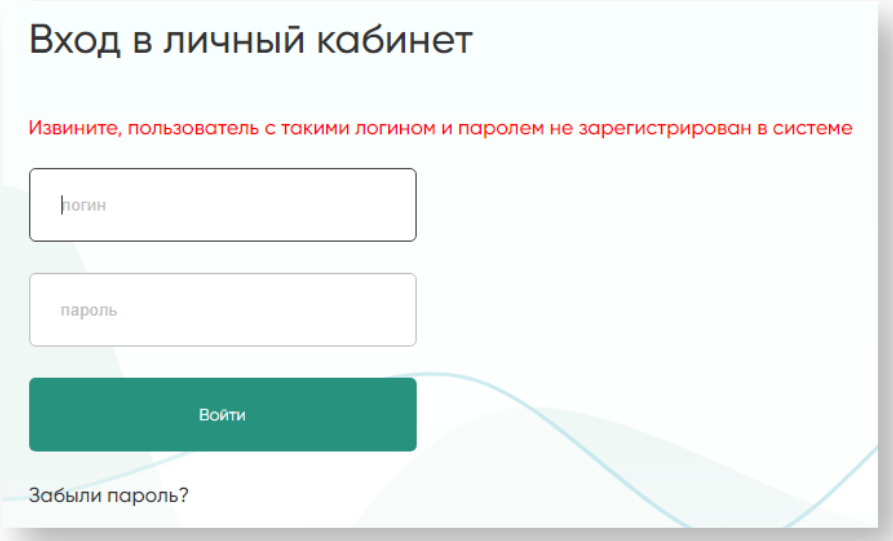

```
@Table({ tableName: 'users' })
export class User extends Model {
  @Column({ primaryKey: true, autoIncrement: true })
  declare id: number;

  @Column({ unique: true, allowNull: false })
  declare email: string;

  @Column({ allowNull: false })
  declare password: string;
```

Аутентификация с Redis *

- Чтобы предоставить доступ пользователю в приложении и наделить его правами, мы реализуем авторизацию



- Если у нас один вид пользователей – одна роль, нам достаточно просто проверить аутентифицирован ли он

```
async login(email: string, password: string) {
  const user = await this.userService.findByEmail(email);
  if (!user || user.password !== password) {
    throw new UnauthorizedException('Invalid credentials');
  }

  const sessionId = uuidv4();
  await this.sessionService.create(sessionId, user.id);
  return { sessionId, user: { id: user.id, email: user.email, role: user.role } };
}
```

Авторизация. Ролевая модель *

- Для каждого пользователя в нашей БД мы указываем его **роль**
- Это может быть поле в таблице пользователей, отдельная таблица или набор таблиц. Так мы можем разделить функционал по отдельным ролям
- В наших функциях указываем конкретные разрешения, которые требуются пользователям для выполнения действий
- Также мы выполняем **идентификацию**, чтобы отображать заявки **этого** пользователя, а для **модератора** - **все заявки**

```
@Get(':id')
@UseGuards(SessionGuard)
async findOne(@Param('id') id: string, @Req() req: Request & { session: { userId: number } }) {
  const request = await this.requestService.findById(+id);
  if (!request) {
    throw new NotFoundException('Request not found');
  }
  // Проверка: только свой или модератор (опционально)
  if (request.authorId !== req.session.userId) {
    throw new ForbiddenException('Access denied');
  }
  return request;
}
```

Авторизация Резюме *

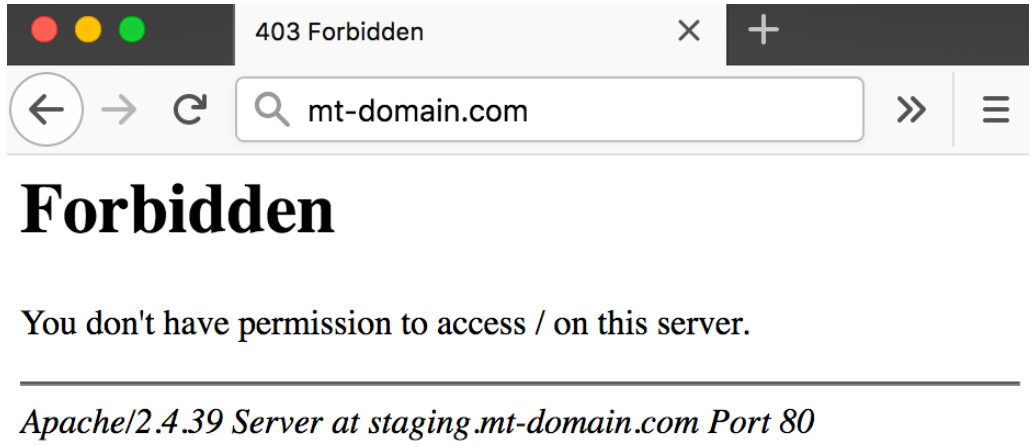
Соответственно в методах, в которых нужно проверить имеет ли пользователя доступ к запрашиваемой информации, мы должны:

- взять из запроса куки (через ssid = request.COOKIES["session_id"]) или JWT из authorization для Golang
- Посмотреть, есть ли в хранилище сессий такая запись, и достать идентификатор пользователя (session_storage.get(ssid)). В Golang проверить JWT и из него id пользователя
- проверить, можно ли данному пользователю смотреть запрошенную информацию через permissions (зависит от бизнес-логики вашего проекта)

- В **методе аутентификации** обратно во фронтенд необходимо отправить **признак модератора**, чтобы изменить представление приложения

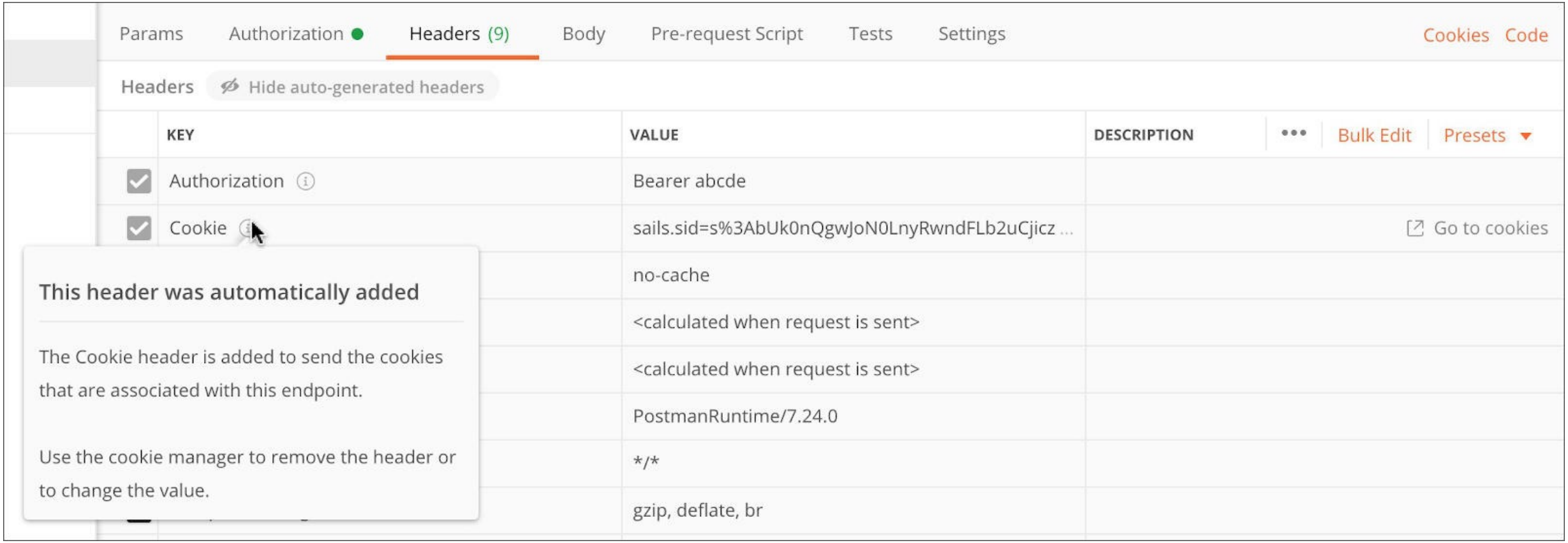
Авторизация. Ограничения на бэкенде*

- Чтобы ограничить неавторизованным пользователем доступ к контенту, создадим view и добавим туда authentication_classes и permission_classes
- Получаем код 403 (у пользователя нет прав), 401 (пользователь не аутентифицирован)



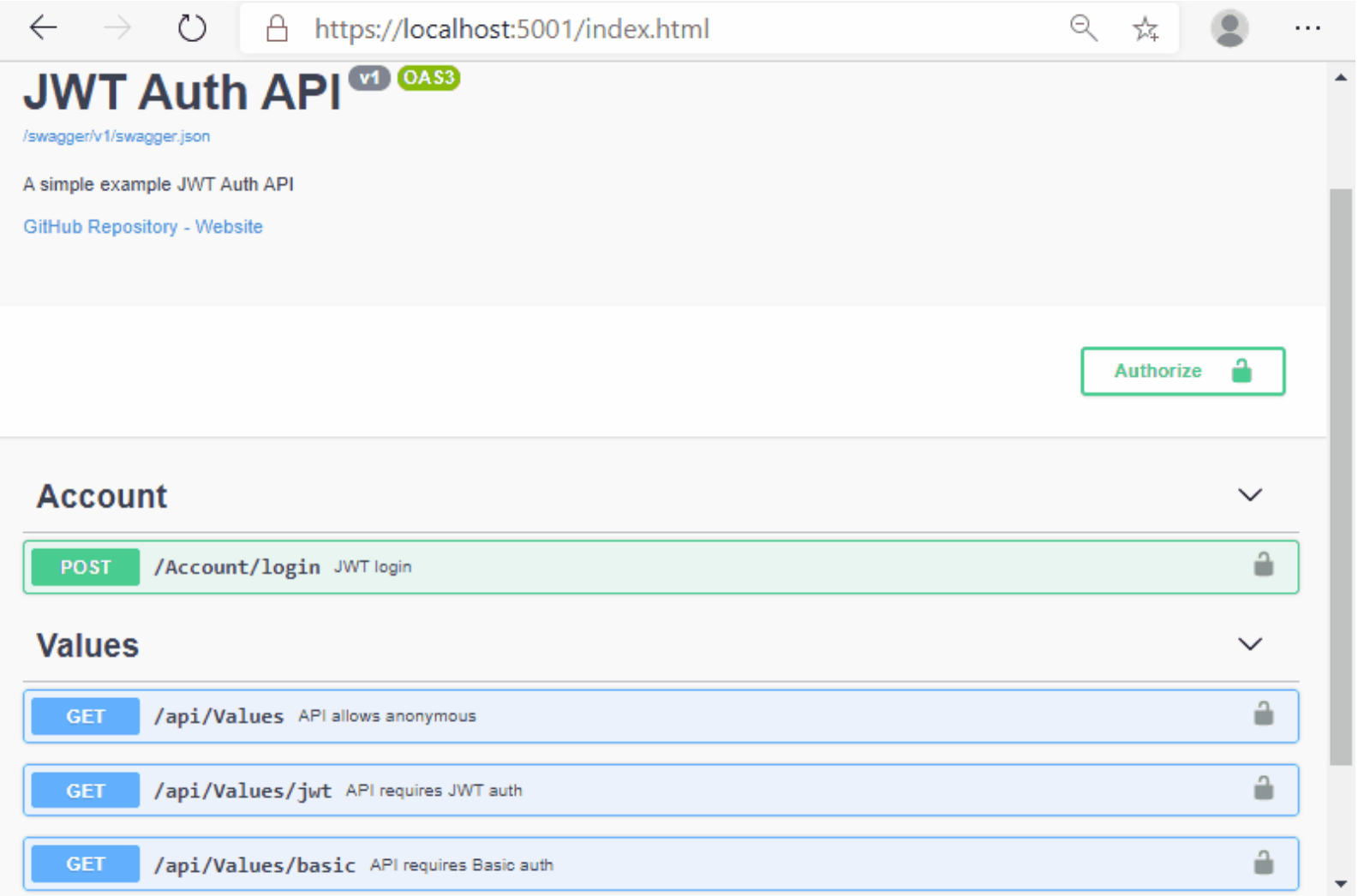
Postman

- При тестировании наши куки (или токен JWT) указываем в заголовках каждого запроса **вручную** сами
- **Не используем** вкладку Authorization и меню Cookie при демонстрации

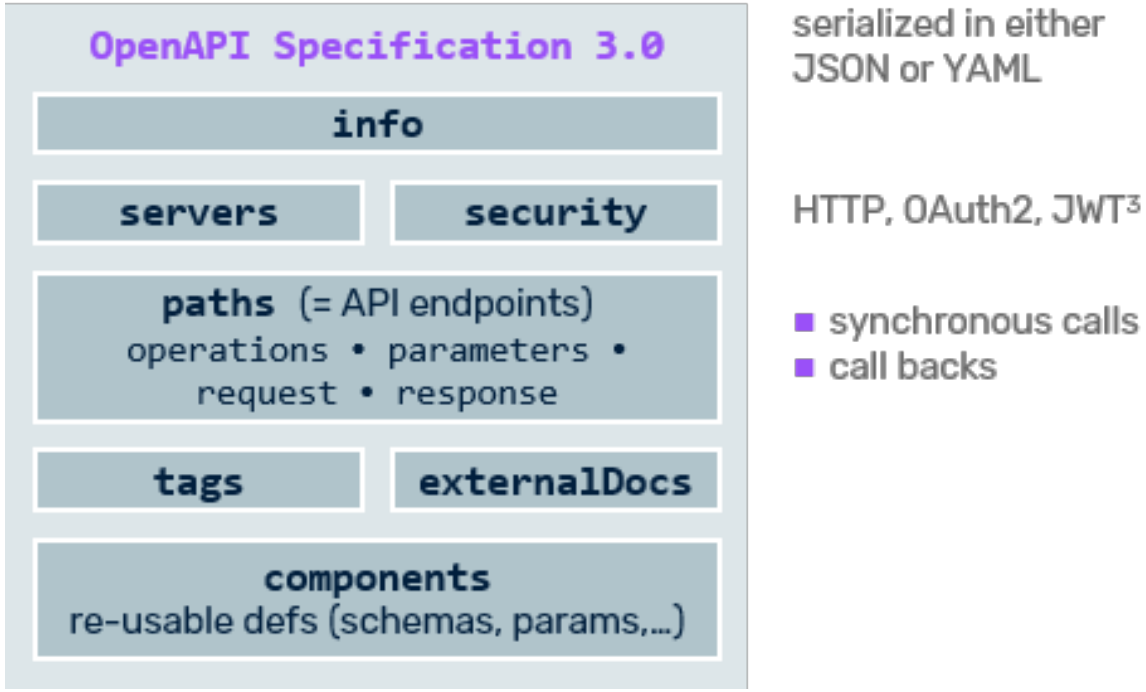


Swagger

- Swagger – это фреймворк для спецификации RESTful API.
- Swagger UI дает возможность интерактивно просматривать спецификацию и отправлять запросы



OpenAPI



- **The OpenAPI Specification** (изначально известная как *Swagger Specification*)
- Формализованная спецификация и экосистема множества инструментов, предоставляющая интерфейс между front-end системами, кодом библиотек низкого уровня и коммерческими решениями в виде API.
- Вместе с тем, спецификация построена таким образом, что не зависит от языков программирования, и удобна в использовании как человеком, так и машиной

Добавление Swagger

```
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
```

- Используем пакет для swagger
- Подключаем swagger в url, обработчики появятся в swagger автоматически
- Лучше описать dto для всех методов
- По этому адресу будет json файл. Мы будем использовать его для генерации кода фронтенда

```
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);
```

```
import { ApiProperty } from '@nestjs/swagger';
export class LoginDto {
  @ApiProperty({ example: 'user@test.com' })
  email: string;
  @ApiProperty({ example: '123' })
  password: string;
}
```

```
@Post('login')
async login(
  @Body() dto: LoginDto,
  @Res({ passthrough: true }) res: express.Response,
) {
  const { sessionId, user } = await this.authService.login(dto.email, dto.password);
  res.cookie('sessionId', sessionId, {
```

http://127.0.0.1:3000/api/?format=openapi

Просмотр

http://localhost:3000/api

- По данной ссылке доступен наш swagger
- Теперь здесь мы можем протестировать все наши методы
- Из вкладки **Application** мы копируем значение **куки** для **Postman**

Qwen Chat | Profile | Swagger UI | Editing nestjs_methoda/READM | окак - По

http://localhost:3000/api?spm=a2ty_o01.29997173.0.0.65865171acRHgt#/Auth/AuthController_login

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3000/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "okak@okak.com",
  "password": "1234"
}'
```

Request URL

```
http://localhost:3000/api/auth/login
```

Инспектор | Консоль | Отладчик | Сеть | Стили | Профайлер | Память | Хранилище | Поддержка доступности | Приложение

Поиск URL | Все | HTML | CSS | JS | XHR | Шрифты | Изображения | Медиа | WS | Прочее | Отключить кеш | Без ограничения

Статус	Метод	Домен	Файл	Инициатор	Тип	Передано	Размер
201	POST	localhost:3000	login	swagger-ui-bundle.js:2...	json	426 б	55 б

1 запрос | 55 б / 426 б передано | Передано за: 45 мс

Заголовки | Куки | Запрос | Ответ | Тайминги | Стек вызовов

Поиск заголовков | Блокировать | Повторить отправку

POST http://localhost:3000/api/auth/login

Состояние	201 Created
Версия	HTTP/1.1
Передано	426 б (размер 55 б)
Referrer policy	strict-origin-when-cross-origin
Приоритет запроса	Highest
Поиск в DNS	Система

Заголовки ответа (371 б) | Необработанные

Connection: keep-alive
Content-Length: 55
Content-Type: application/json; charset=utf-8
Date: Sun, 28 Dec 2025 20:08:26 GMT
ETag: W/"37-g4cHluqQd8HTxSSokK3OaFrh0pI"
Keep-Alive: timeout=5
Set-Cookie: sessionId=ee1eea34-1e40-469c-8f96-8089a74f6883; Max-Age=3600; Path=/; Expires=Sun, 28 Dec 2025 21:08:26 GMT; HttpOnly

Поиск в консоли | Ошибки | Предупреждения | Инфо | Лог | Отладка | CSS | XHR | Запросы

XHR POST http://localhost:3000/api/auth/login [HTTP/1.1 201 Created 45ms]